RETAIL ML

Système de Prédiction du Churn

Architecture, Modèles & Pipeline ML de Rétention

Plan de la Présentation

01-03

Architecture, Structure & Pipeline de données

04-05

Modèles ML, Preprocessing & Performance

06-07

Interface Flask, API & Guide d'Utilisation

08-10

Architecture technique, Points forts & Futur

01 / Vue d'Ensemble

Mission du Projet

Prédire les risques de départ client (Churn) et segmenter la clientèle pour des actions de rétention ultra-ciblées.

Domaine : Retail / E-commerce

Approche : Multi-modèles (Classification, Clustering, Régression).

Stack Technique

- Python (Scikit-Learn, XGBoost)
- Flask (API & Web Dashboard)
- Pandas, NumPy, PCA

02 / Structure du Projet

- app/ : Serveur Flask et Templates
- data/ : Données Raw et Processed
- models/ : Fichiers .pkl sérialisés
- src/ : Scripts Preprocessing et Train
- reports/ : Visualisations ROC et Importance

app/

Interface web et API

```
├── api/app.py ← Serveur Flask
└── front/src/ ← Templates HTML
```

data/

Données et datasets

```
├── raw/customers.csv ← Données brutes
├── processed/ ← Données nettoyées
└── train_test/ ← Splits
```

models/

Modèles sérialisés (.pkl)

churn_rf.pkl
churn_lr.pkl
churn_xgb.pkl
cluster_kmeans.pkl
reg_gb_best.pkl
preprocessor.pkl

reports/

Visualisations et analyses

roc_curves.png
feature_importance.png
regression_comparison.png
actual_vs_predicted.png

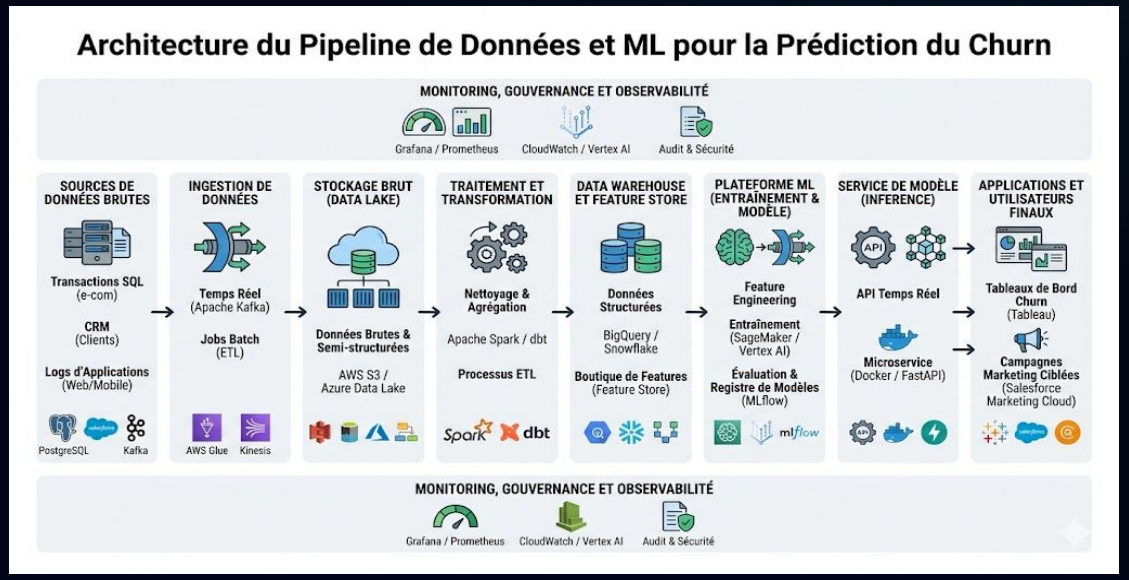
src/

Scripts Python

preprocessing.py ← DataPreprocessor
train_model.py ← Classification
clustering.py ← Clustering + PCA
regression.py ← Régression
predict.py ← CLI

03 / Architecture du Pipeline

- **RAW CSV** : Chargement des données brutes.
- **DataPreprocessor** : Nettoyage (Outliers) et transformation.
- **Feature Engineering** : Calcul RFM et ratios temporels.
- **Modélisation** : Triple branche (Classification, Clustering, Régression).



05 / Pipeline Preprocessing

- **Nettoyage :** Filtrage des tickets > 999 et satisfaction ≤ 50.
- **Temporal Extraction :** Extraction Year, Month, Day, Weekday.
- **Feature Engineering :** Calcul MonetaryPerDay et IsDormant.
- **Imputation :** Utilisation de SimpleImputer(median) pour les valeurs manquantes.
- **Scaling :** StandardScaler pour assurer l'équilibre des poids.
- **Zéro Fuite :** Pattern fit() sur Train uniquement, transform() sur les deux.

04 / Classification (Churn)

82%

Taux de Rappel (Recall)

Performance XGBoost

Le modèle identifie 82% des clients qui vont réellement partir, minimisant les opportunités de rétention manquées.

- **Précision** : 86%
- **AUC-ROC** : 0.91

Comparaison des Modèles

Type	Modèle	Objectif Principal	Status
Classification	**XGBoost**	Performance & Régularisation	Sélectionné
Classification	Random Forest	Interprétabilité & Baseline	Backup
Clustering	K-Means + PCA	Segmentation Clientèle	Actif
Régression	Gradient Boosting	Prédiction MonetaryTotal	Actif

04 / Clustering & Segments

Segments Identifiés

- Champions (15%)
- Fidèles (30%)
- À Risque (30%)
- Dormants (25%)

04 / Régression (Valeur Future)

Prédiction de la Customer Lifetime Value (CLV) pour prioriser les budgets.

06 / Interface Web & API

- **Dashboard :** Vue KPIs en temps réel.
- **Endpoint /predict :** API REST JSON.
- **Batch Process :** Upload CSV massif.
- **Healthcheck :** Monitoring d'état API.

06 / Gestion des Risques

Faible (<40%)

Fidélisation & Upsell. Offres premium.

Élevé (40-70%)

Réengagement & Enquêtes satisfaction.

Critique (>70%)

Actions immédiates. Contact direct CRM.

07 / Utilisation & Commandes

```
>_ python src/preprocessing.py (Nettoyage)
>_ python src/train_model.py (Classification)
>_ python src/predict.py --input data/raw/customers.csv
>_ cd app/api && python app.py (Lancer Flask)
```

Architecture Technique (Flux)

Client

Requête (Web/API)

Flask

Traitement & Routage

Models

Inférence .pkl

Response

Prédiction & Risque

09 / Points Forts

- **Pipeline Robuste :** fit/transform sans fuite.
- **Multi-modèles :** Churn + Segments + CLV.
- **Sérialisation :** Preprocessor.pkl prêt prod.
- **Extensible :** Architecture modulaire isolée.

Merci !

Questions & Discussions